Working with nested Inner class

package com.rest.nest.inner_class

Inner class is a class defined within another class.

It is some which are not declared at class level.

Inner class is a class defined inside a base but outside of the method then it is called non static inner class

Example

```
public class Outer
{
    public class Inner
    {
        // Instance variables
        private int i = 10;

        // Constructor
        private int x = 0;
        public Inner(int pi)
        {
            System.out.println("Inside Inner constructor");
            System.out.println("Value of x = " + pi);
            x = pi;
        }

        // Instance methods
        public void show()
        {
            System.out.println("Inside show method");
            System.out.println("Value of i = " + i);
            System.out.println("Value of x = " + x);
        }
    }

    // Instance variables
    private int i = 20;

    // Constructor
    public Outer(int pi)
    {
        System.out.println("Inside Outer constructor");
        System.out.println("Value of i = " + pi);
        i = pi;
    }

    // Instance methods
    public void show()
    {
        System.out.println("Inside show method");
        System.out.println("Value of i = " + i);
        Inner obj = new Inner(i);
        obj.show();
    }
}
```

Variable Shadow in Inner class

If the local variable and instance variable same as inner than inner class variable will be prefixed with inner class object (Variable Shadow because or the same scope). If we want to access outer class variable then we have to use outer class object (Variable Shadowing).

package com.rest.nest.inner_class.variable_shadow

```
class Outer
{
    private int x = 5;

    public void method()
    {
        System.out.println("Inner class method");
        System.out.println("Value of x = " + x);
        Inner obj = new Inner();
        obj.method();
    }

    class Inner
    {
        private int x = 10;

        public void method()
        {
            System.out.println("Value of x = " + x);
        }
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer obj = new Outer();
        obj.method();
    }
}
```

package com.rest.nest.inner_class.forbidden

```
class Outer
{
    private int i = 10;

    public void method()
    {
        System.out.println("Inner class method");
        System.out.println("Value of i = " + i);
        Inner obj = new Inner();
        obj.method();
    }

    class Inner
    {
        private int x = 20;

        public void method()
        {
            System.out.println("Value of x = " + x);
            System.out.println("Value of i = " + i);
        }
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer obj = new Outer();
        obj.method();
    }
}
```

Note : Inner class shares their scope with outer class. Inner class shares data with outer class.

package com.rest.nest.inner_class.share

```
class Outer
{
    private int i = 10;
    private String name = "Rahul";

    public void method()
    {
        System.out.println("Inside method");
        System.out.println("Value of i = " + i);
        System.out.println("Value of name = " + name);
        Inner obj = new Inner();
        obj.method();
    }

    class Inner
    {
        public void method()
        {
            System.out.println("Inside Inner method");
            System.out.println("Value of i = " + i);
            System.out.println("Value of name = " + name);
        }
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer obj = new Outer();
        obj.method();
    }
}
```

Note: Inner class is private and Outer class is public

Outer class can access Inner class data.

package com.rest.nest.inner_class.access

```
class Outer
{
    private int i = 10;

    public void method()
    {
        System.out.println("Inside Outer method");
        Inner obj = new Inner();
        obj.method();
    }

    class Inner
    {
        public void method()
        {
            System.out.println("Inside Inner method");
            System.out.println("Value of i = " + i);
        }
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer obj = new Outer();
        obj.method();
    }
}
```

Note: Outer class can access Inner class data.

Inner class can access Outer class data.

package com.rest.nest.inner_class.access_both

```
class Outer
{
    private int i = 10;

    public void method()
    {
        System.out.println("Inside Outer method");
        Inner obj = new Inner();
        obj.method();
    }

    class Inner
    {
        public void method()
        {
            System.out.println("Inside Inner method");
            System.out.println("Value of i = " + i);
        }
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer obj = new Outer();
        obj.method();
    }
}
```

package com.rest.nest.inner_class.

```
class Outer
{
    private int i = 10;

    inner InnerClass
    {
        public void display()
        {
            System.out.println("Inside display method");
            System.out.println("Value access outer private var", +i);
        }
    }

    public void getInstance()
    {
        System.out.println("Inside getInstance");
        InnerClass obj = new InnerClass();
        obj.display();
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer.InnerClass obj = new Outer().new InnerClass();
        obj.display();
        new Outer().getInstance();
    }
}
```

package com.rest.nest.inner_class.

```
class Outer
{
    private int i = 10;

    inner InnerClass
    {
        public void display()
        {
            System.out.println("Inside display method");
            System.out.println("Value access outer private var", +i);
        }
    }

    public void getInstance()
    {
        System.out.println("Inside getInstance");
        InnerClass obj = new InnerClass();
        obj.display();
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer.InnerClass obj = new Outer().new InnerClass();
        obj.display();
        new Outer().getInstance();
    }
}
```

package com.rest.nest.inner_class.

```
class Outer
{
    public static void main()
    {
        System.out.println("Inside Outer method");
        InnerClass obj = new InnerClass();
        obj.display();
    }

    inner InnerClass
    {
        public void display()
        {
            System.out.println("Inside display method");
            System.out.println("Value access outer private var", +i);
        }
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer.main();
    }
}
```

Static Inner class

Note : In non static inner class we can declare with private, default, protected, public. But in static inner class only static declare.

In static inner class we can declare or access static variable.

If we declare a class inside an outer class with static modifier then it is called static nested class.

Example

```
public class Outer
{
    static class Inner {static nested class}
}
```

In order to access the static nested class, Outer class object is not required.

Static nested class can access only static variable. Static nested class can access static data of outer class but not other field if static nested class inner class contains only static member then inner class object is not required.

If a static nested class has static member and is not private non static member of Outer class.

package com.rest.static.nested_class.

```
// Static nested inner class
class Outer
{
    static class Inner {static nested inner class}

    public void display()
    {
        System.out.println("Inside Outer method");
        Inner obj = new Inner();
        obj.display();
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer.Inner obj = new Outer.Inner();
        obj.display();
        new Outer().display();
    }
}
```

package com.rest.nest.inner_class.

```
class Outer
{
    static class InnerClass
    {
        int i = 10;

        private static int[] arr = {1, 2, 3};

        public void display()
        {
            System.out.println("Inside display method");
            System.out.println("Value access outer private var", +i);
            System.out.print("Value of array");
            for (int i : arr)
            {
                System.out.print(" " + i);
            }
            new Outer().display();
        }
    }

    public void display()
    {
        System.out.println("Inside Outer method");
        InnerClass obj = new InnerClass();
        obj.display();
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer.InnerClass obj = new Outer.InnerClass();
        obj.display();
        new Outer().display();
    }
}
```

package com.rest.nest.inner_class.

```
class SuperClass
{
    int i = 10;

    private class InnerClass
    {
        int i = 20;

        public void display()
        {
            System.out.println("Inside display method");
            System.out.print("Value access super private var", +i);
            System.out.print("Value access inner private var", +i);
        }
    }

    public void getInstance()
    {
        System.out.print("Inside getInstance");
        InnerClass obj = new InnerClass();
        obj.display();
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        SuperClass.InnerClass obj = new SuperClass().new InnerClass();
        obj.display();
        new SuperClass().getInstance();
    }
}
```

package com.rest.nest.inner_class.

```
class SuperClass
{
    int i = 10;

    private class InnerClass
    {
        int i = 20;

        public void display()
        {
            System.out.print("Inside display method");
            System.out.print("Value access super private var", +i);
            System.out.print("Value access inner private var", +i);
        }
    }

    public void getInstance()
    {
        System.out.print("Inside getInstance");
        InnerClass obj = new InnerClass();
        obj.display();
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        SuperClass.InnerClass obj = new SuperClass().new InnerClass();
        obj.display();
        new SuperClass().getInstance();
    }
}
```

package com.rest.nest.inner_class.

```
class SuperClass
{
    int i = 10;

    private class InnerClass
    {
        int i = 20;

        public void display()
        {
            System.out.print("Inside display method");
            System.out.print("Value access super private var", +i);
            System.out.print("Value access inner private var", +i);
            System.out.print("Value access outer private var", +i);
        }
    }

    public void getInstance()
    {
        System.out.print("Inside getInstance");
        InnerClass obj = new InnerClass();
        obj.display();
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        SuperClass.InnerClass obj = new SuperClass().new InnerClass();
        obj.display();
        new SuperClass().getInstance();
    }
}
```

package com.rest.nest.inner_class.

```
class Outer
{
    private int i = 10;

    private class InnerClass
    {
        int i = 20;

        public void display()
        {
            System.out.print("Inside display method");
            System.out.print("Value access super private var", +i);
            System.out.print("Value access inner private var", +i);
            System.out.print("Value access outer private var", +i);
            System.out.print("Value access outer private var", +i);
        }
    }

    public void getInstance()
    {
        System.out.print("Inside getInstance");
        InnerClass obj = new InnerClass();
        obj.display();
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer.InnerClass obj = new Outer().new InnerClass();
        obj.display();
        new Outer().getInstance();
    }
}
```

Local Inner class

Local Inner class is a class which is declared inside the method body then it is called Local Inner class.

Example

```
public class Outer
{
    public void method()
    {
        class Local {Local Inner class}
    }
}
```

The scope of Method local inner class is within the same method body that means it will be executed in the same method block if there.

We can't access or use the method local inner class outside of the method.

Method local inner class is accessible if it is declared in same parameter within the method.

We can't apply any kind of access modifier on local inner class except abstract and final.

package com.rest.nest.inner_class.method_local

```
class Outer
{
    private String s = "Inner class private data";

    public void method()
    {
        // Local class
        class Local implements Inner, {//Local is anonymous Inner}
        {
        public void display()
        {
            System.out.print("Inside Local");
            System.out.print("Inside Inner");
            System.out.print("Inside Outer");
            System.out.print("Inside Outer private data", s);
        }
        }

        Local obj = new Local();
        obj.display();
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer obj = new Outer();
        obj.method();
    }
}
```

package com.rest.nest.inner_class.method_local.

```
class Outer
{
    private int i = 10;

    public void method()
    {
        // Local class
        class Local
        {
            public void display()
            {
                System.out.print("Inside Local");
                System.out.print("Value access outer private var", +i);
                System.out.print("Value access outer private var", +i);
                System.out.print("Value access outer private var", +i);
                System.out.print("Value access outer private var", +i);
            }
        }

        Local obj = new Local();
        obj.display();
    }
}

public class Test
{
    public static void main(String[] pi)
    {
        Outer obj = new Outer();
        obj.method();
    }
}
```

Anonymous Inner class

If we declare inner class inside a method without any name then it is called Anonymous inner class.

The main purpose of anonymous inner class is extend a base (IN is implement an interface that anonymous inner class is implement).

Anonymous inner class syntax is very complicated and it is declared and created both at the same time at the time of declaration of anonymous inner class.

Inner class object only can access the anonymous inner class so we can represent as a singleton.

If anonymous inner class is extend a class as well as implement many number of interfaces but for anonymous inner class we can't write any constructor because we don't know what is the name of the class.

Anonymous inner class can access only static data, static field, static method and static methods, we can't access constructor as well as we can't declare as an abstract class.

package com.rest.nest.anonymous_inner_class.

```
interface Sayhi
{
    void sayhi();
}

public class AnonymousInner
{
    public static void main(String[] pi)
    {
        Sayhi obj = new Sayhi();
        obj.sayhi();
    }
}
```

package com.rest.nest.anonymous_inner_class.

```
interface Sayhi
{
    void sayhi();
}

class SuperClass
{
    public SuperClass(int pi)
    {
        System.out.print("Inside SuperClass constructor");
        System.out.print(" " + pi);
    }
}

public class AnonymousInner
{
    public static void main(String[] pi)
    {
        Sayhi obj = new Sayhi();
        obj.sayhi();
        SuperClass sc = new SuperClass(10);
    }
}
```

package com.rest.nest.anonymous_inner_class.

```
interface Sayhi
{
    void sayhi();
}

class SuperClass
{
    public SuperClass(int pi)
    {
        System.out.print("Inside SuperClass constructor");
        System.out.print(" " + pi);
    }
}

public class AnonymousInner
{
    public static void main(String[] pi)
    {
        Sayhi obj = new Sayhi();
        obj.sayhi();
        SuperClass sc = new SuperClass(10);
        System.out.print("Inside this anonymous inner class");
```